SAA - ACTIVIDAD 3.10 - ÁLVARO FERNANDEZ BECERRA

Estudio original de la actividad anterior:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay, classification_report
# Para usar el clasificador GaussianNB:
from sklearn.naive_bayes import GaussianNB

url = "https://archive.ics.uci.edu/ml/machine-learning-databases/heart-disease/processed.cleveland.data"

# Nombres de columnas
columnas = [
    'age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg',
    'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal', 'num'
]

df = pd.read_csv(url, header=None, names=columnas, na_values='?')
df.head()
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
0	63.0	1.0	1.0	145.0	233.0	1.0	2.0	150.0	0.0	2.3	3.0	0.0	6.0	0
1	67.0	1.0	4.0	160.0	286.0	0.0	2.0	108.0	1.0	1.5	2.0	3.0	3.0	2
2	67.0	1.0	4.0	120.0	229.0	0.0	2.0	129.0	1.0	2.6	2.0	2.0	7.0	1
3	37.0	1.0	3.0	130.0	250.0	0.0	0.0	187.0	0.0	3.5	3.0	0.0	3.0	0
4	41.0	0.0	2.0	130.0	204.0	0.0	2.0	172.0	0.0	1.4	1.0	0.0	3.0	0

Pasos siguientes: Generar código con df | New interactive sheet

Haz doble clic (o pulsa Intro) para editar

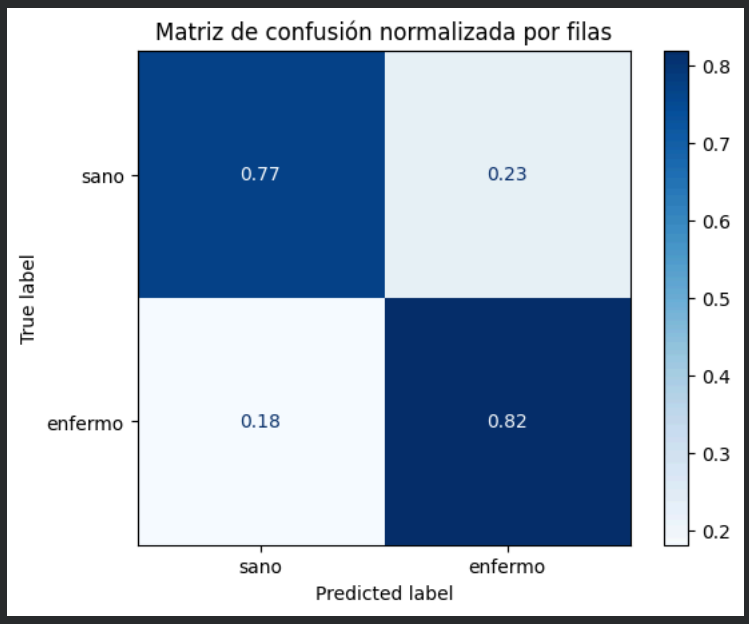
```
# Eliminar filas con nulos
df.dropna(inplace=True)

# Convertimos num a binario donde: 0:sano y valores mayores -> 1:enfermo
for ind, row in df.iterrows():
    if row['num'] > 0:
        df.at[ind, 'num'] = 1

# x y
x = df.drop(['num', 'cp', 'restecg', 'slope', 'thal'], axis=1)
y = df['num']
```

```
RANDOM_SEED = 30

x_train, x_test, y_train, y_test = train_test_split(
  x, y, test_size=0.75, random_state=RANDOM_SEED)

print('Dimensiones de X,Y dataset de entrenamiento: {dimX}, {dimY}.'.format(dimX=x_train.shape, dimY=y_train.shape))
print('Dimensiones de X,Y dataset de validación: {dimX}, {dimY}.'.format(dimX=x_test.shape, dimY=y_test.shape))

# Crear y entrenar el clasificador
clf = GaussianNB()
clf.fit(x_train, y_train)

y_pred = clf.predict(x_test)

# Porcentaje:
print(f"Accuracy en validación: {accuracy_score(y_test, y_pred):.4f}")
```

```
Dimensiones de X,Y dataset de entrenamiento: (74, 9), (74,).
Dimensiones de X,Y dataset de validación: (223, 9), (223,).
Accuracy en validación: 0.7892
```

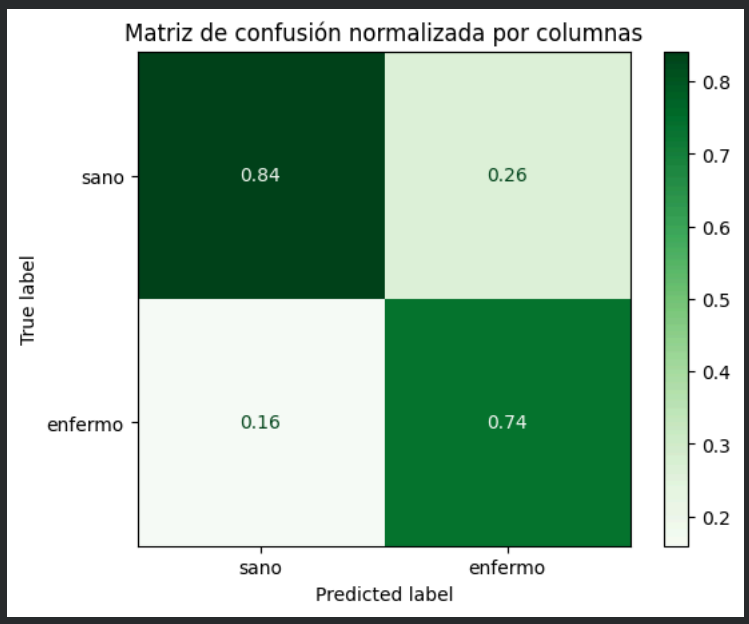
- Matriz de confusión normalizada por filas

```
matriz_filas = confusion_matrix(y_test, y_pred, normalize='true')

disp_filas = ConfusionMatrixDisplay(
    confusion_matrix=matriz_filas,
    display_labels=['sano', 'enfermo']
)
disp_filas.plot(cmap='Blues')
plt.title('Matriz de confusión normalizada por filas')
plt.show()
```



- Matriz de confusión normalizada por columnas

```
matriz_columnas = confusion_matrix(y_test, y_pred, normalize='pred')

disp_columnas = ConfusionMatrixDisplay(
    confusion_matrix=matriz_columnas,
    display_labels=['sano', 'enfermo']
)
disp_columnas.plot(cmap='Greens')
plt.title('Matriz de confusión normalizada por columnas')
plt.show()
```



- Classification Report del dataset:

```
print(classification_report(y_test, y_pred, target_names=['sano', 'enfermo']))
```

```
              precision    recall  f1-score   support
```

```
        sano       0.84      0.77      0.80       124
     enfermo       0.74      0.82      0.78        99

    accuracy                           0.79       223
   macro avg       0.79      0.79      0.79       223
weighted avg       0.79      0.79      0.79       223
```

Estudio actividad 3.10:

-Esta vez vamos a usar pd.get_dummies() para transformar los atributos categoricos a numericos, lo que al hacer el estudio nos debería dar mejores resultados que en la actividad anterior.

```
df2 = pd.read_csv(url, header=None, names=columnas, na_values='?')
df2.dropna(inplace=True)

# # Convertimos num a binario donde: 0:sano y valores mayores -> 1:enfermo
for ind, row in df2.iterrows():
    if row['num'] > 0:
        df2.at[ind, 'num'] = 1

#target
y2 = df2['num']

# Aplicar get_dummies sobre los atributos categoricos
x2 = df2.drop('num', axis=1)
x2 = pd.get_dummies(x2, columns=['cp', 'restecg', 'slope', 'thal'])

x2.head()
```

```
(297, 22)
```

	age	sex	trestbps	chol	fbs	thalach	exang	oldpeak	ca	cp_1.0	...	cp_4.0	restecg_0.0	restecg_1.0	restecg_2.0	s
0	63.0	1.0	145.0	233.0	1.0	150.0	0.0	2.3	0.0	True	...	False	False	False	True	
1	67.0	1.0	160.0	286.0	0.0	108.0	1.0	1.5	3.0	False	...	True	False	False	True	
2	67.0	1.0	120.0	229.0	0.0	129.0	1.0	2.6	2.0	False	...	True	False	False	True	
3	37.0	1.0	130.0	250.0	0.0	187.0	0.0	3.5	0.0	False	...	False	True	False	False	
4	41.0	0.0	130.0	204.0	0.0	172.0	0.0	1.4	0.0	False	...	False	False	False	True	

5 rows × 22 columns

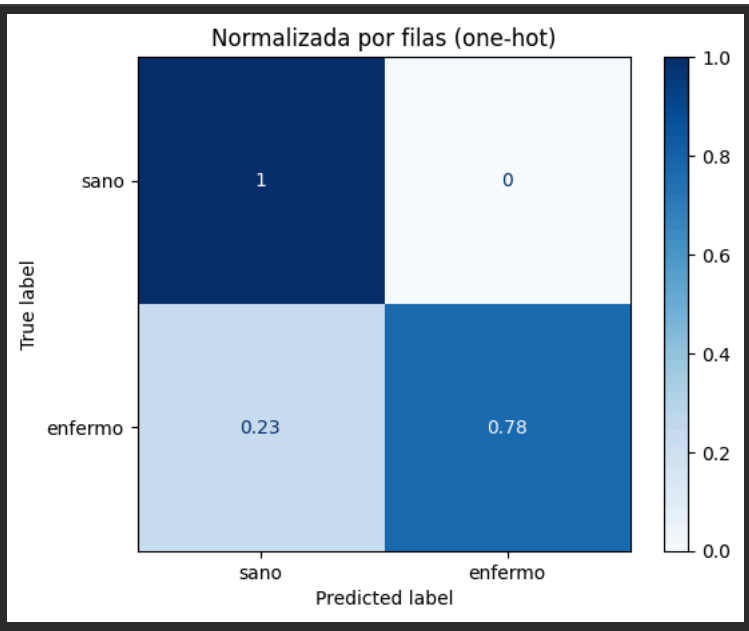
```
# Entremanmiento y validacion
RANDOM_SEED = 40
x2_train, x2_test, y2_train, y2_test = train_test_split(
    x2, y2, train_size=0.75, random_state=RANDOM_SEED
)

clf2 = GaussianNB()
clf2.fit(x2_train, y2_train)

y2_pred = clf2.predict(x2_test)

accuracy_clf2 = accuracy_score(y2_test, y2_pred)
print(f"Accuracy GaussianNB con one-hot encoding: {accuracy_clf2:.4f}")
print(f"Accuracy GaussianNB sin one-hot encoding: {accuracy_score(y_test, y_pred):.4f}" )
```

```
Accuracy GaussianNB con one-hot encoding: 0.8800
Accuracy GaussianNB sin one-hot encoding: 0.7892
```

- Observamos que al tener ahora todos los atributos como numericos el porcentaje de acierto ha aumentado significativamente

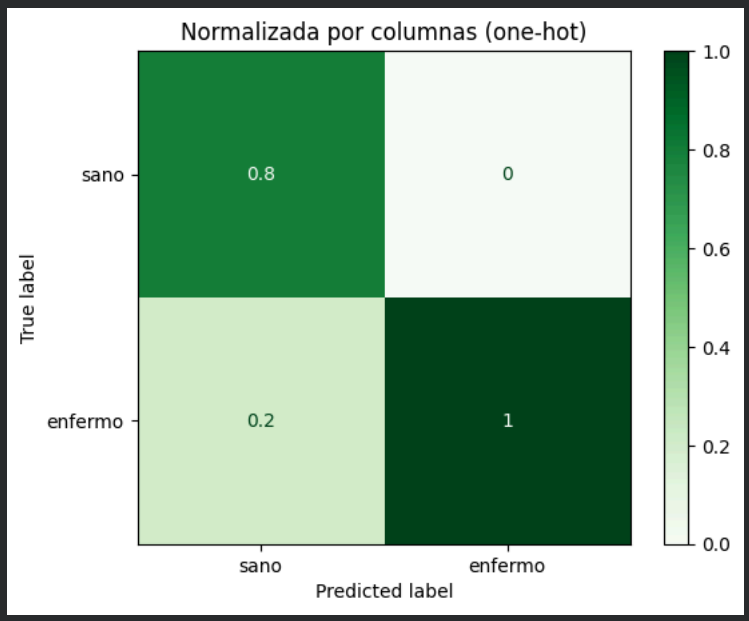
Matriz de confusion normalizada por filas:

```
matriz2_filas = confusion_matrix(y2_test, y2_pred, normalize='true')
ConfusionMatrixDisplay(matriz2_filas, display_labels=['sano', 'enfermo']).plot(cmap='Blues')
plt.title('Normalizada por filas (one-hot)')
plt.show()
```



Matriz de confusion normalizada por columnas:

```
matriz2_columnas = confusion_matrix(y2_test, y2_pred, normalize='pred')
ConfusionMatrixDisplay(matriz2_columnas, display_labels=['sano', 'enfermo']).plot(cmap='Greens')
plt.title('Normalizada por columnas (one-hot)')
plt.show()
```




```
print("REPORTE CON pd.get_dummies: \n")
print(classification_report(y2_test, y2_pred, target_names=['sano', 'enfermo']))

print("\nREPORTE ORIGINAL SIN pd.get_dummies: \n")
print(classification_report(y_test, y_pred, target_names=['sano', 'enfermo']))
```

```
REPORTE CON pd.get_dummies:

              precision    recall  f1-score   support

        sano       0.80      1.00      0.89        35
     enfermo       1.00      0.78      0.87        40

    accuracy                           0.88        75
   macro avg       0.90      0.89      0.88        75
weighted avg       0.90      0.88      0.88        75


REPORTE ORIGINAL SIN pd.get_dummies:

              precision    recall  f1-score   support

        sano       0.84      0.77      0.80       124
     enfermo       0.74      0.82      0.78        99
```

```
    accuracy                           0.79       223
   macro avg       0.79      0.79      0.79       223
weighted avg       0.79      0.79      0.79       223
```

- Gracias a que el modelo puede trabajar con las columnas categoricas convertidas numericamente observamos una clara mejora.
- Se puede sacar como conclusión que evidentemente los atributos categoricos, que antes se descartaban, aportaban información y datos utiles al modelo.